

Contrat d'API Complet - Marketplace Microservices

Version Mise à Jour avec Architecture Hybride SQL/NoSQL

Contexte et Conventions Générales

- Architecture : API Gateway unique + microservices indépendants
- Services : users:3001 (SQL), catalog:3002 (NoSQL), cart:3004 (NoSQL), orders:3003 (NoSQL), payments:3005 (NoSQL), addresses:3006 (NoSQL)
- Format : JSON bodies uniquement
- Authentification : JWT via Authorization: Bearer <token>, Gateway ajoute X-User-Id, X-User-Role
- Rôles : client, vendeur, admin
- Pagination : ?page=0&limit=20&sort=price:desc
- Erreurs : { "error": "message", "code": "ERR_INVALID_INPUT" }
- Base URL externe : http://localhost:3000/api

1. API Externe (Front-end → API Gateway)

1.1 Authentification [SQL PostgreSQL - users:3001]

POST /auth/register

Description : Création de compte utilisateur

Body :

```
{  
  "username": "string (required, unique, 3-30 chars)",  
  "password": "string (required, min 8 chars)",  
  "email": "string (required, valid email)",  
  "role": "string (optional, default 'client')"  
}
```

Response : 201 -

```
{  
  "userId": "uuid",  
  "username": "string",  
  "email": "string",  
  "role": "string"  
}
```

Erreurs : 400 (validation), 409 (username/email exists)

Autorisation : Aucune (public)

POST /auth/login

Description : Authentification et génération JWT

Body :

```
{  
  "username": "string (required)",  
  "password": "string (required)"  
}
```

Response : 200 -

```
{  
  "token": "string (JWT)",  
  "userId": "uuid",  
  "role": "string",  
  "expiresIn": 86400  
}
```

Erreurs : 401 (invalid credentials)

Autorisation : Aucune (public)

GET /auth/me

Description : Profil utilisateur connecté

Body : none

Response : 200 -

```
{  
  "userId": "uuid",  
  "username": "string",  
  "email": "string",  
  "role": "string",  
  "createdAt": "ISO date"  
}
```

Erreurs : 401 (no token)

Autorisation : Token requis

POST /auth/logout

Description : Invalidation session (blacklist token)

Body : none

Response : 204 No Content

Erreurs : 401 (no token)

Autorisation : Token requis

1.2 Utilisateurs [SQL PostgreSQL - users:3001]

GET /users/:userId

Description : Consulter profil utilisateur

Params : userId (uuid, required)

Body : none

Response : 200 -

```
{
  "userId": "uuid",
  "username": "string",
  "email": "string",
  "role": "string",
  "createdAt": "ISO date",
  "stats": {
    "ordersCount": number,
    "productsCount": number
  }
}
```

Erreurs : 403 (access denied), 404 (not found)

Autorisation : Token requis (client: seulement son compte, admin/vendeur: tous)

PUT /users/:userId

Description : Modifier profil utilisateur

Params : userId (uuid, required)

Body :

```
{
  "username": "string (optional)",
  "email": "string (optional)",
```

"password": "string (optional, min 8)"

}

Response : 200 -

{

"userId": "uuid",

"username": "string",

"email": "string",

"role": "string"

}

Erreurs : 400 (validation), 403 (not owner or admin), 409 (conflict)

Autorisation : Token requis (propriétaire ou admin)

PUT /users/:userId/role

Description : Changer rôle utilisateur

Params : userId (uuid, required)

Body :

{

"role": "string (required, 'client'|'vendeur'|'admin')"

}

Response : 200 -

{

"userId": "uuid",

"role": "string"

}

Erreurs : 400 (invalid role), 403 (not admin), 404 (not found)

Autorisation : Token requis, rôle admin

DELETE /users/:userId

Description : Désactivation compte (soft delete)

Params : userId (uuid, required)

Body : none

Response : 204 No Content

Erreurs : 403 (not owner or admin), 404 (not found)

Autorisation : Token requis (propriétaire ou admin)

1.3 Catalogue Produits [NoSQL MongoDB - catalog:3002]

POST /products

Description : Créer nouveau produit

Body :

```
{
  "name": "string (required, 1-200 chars)",
  "description": "string (optional, max 2000)",
  "price": "number (required, >0)",
  "currency": "string (optional, default 'EUR')",
  "stock": "number (required, >=0)",
  "categoryId": "uuid (required)",
  "images": ["array<string> (optional)"],
  "attributes": "object (optional)"
}
```

Response : 201 -

```
{
  "productId": "uuid",
  "name": "string",
  "description": "string",
  "price": number,
  "currency": "string",
  "stock": number,
  "categoryId": "uuid",
  "sellerId": "uuid",
  "createdAt": "ISO date",
  "images": ["array"],
  "attributes": "object"
}
```

Erreurs : 400 (validation), 403 (not vendeur/admin), 404 (category not found)

Autorisation : Token requis, rôle vendeur ou admin

GET /products

Description : Recherche produits paginée

Query : q=string (search), categoryId=uuid, minPrice=number, maxPrice=number, sellerId=uuid, available=true, page=0, limit=20, sort=price:asc|desc|newest

Body : none

Response : 200 -

```
{
  "products": [
    {
      "productId": "uuid",
      "name": "string",
      "price": number,
      "currency": "string",
      "stock": number,
      "images": ["array"],
      "seller": {
        "userId": "uuid",
        "username": "string"
      }
    }
  ],
  "total": number,
  "page": number,
  "limit": number
}
```

Erreurs : 400 (invalid filters)

Autorisation : Aucune (public)

GET /products/:productId

Description : Détails produit complet

Params : productId (uuid, required)

Body : none

Response : 200 -

```
{
  "productId": "uuid",
  "name": "string",
  "description": "string",
  "price": number,
  "currency": "string",
  "stock": number,
  "categoryId": "uuid",
  "sellerId": "uuid",
  "seller": {
    "username": "string"
  },
  "images": ["array"],
  "attributes": "object",
  "createdAt": "ISO date",
  "avgRating": number,
  "reviewsCount": number
}
```

Erreurs : 404 (not found)

Autorisation : Aucune (public)

PUT /products/:productId

Description : Modifier produit (propriétaire uniquement)

Params : productId (uuid, required)

Body :

```
{
  "name": "string (optional)",
  "description": "string (optional)",
  "price": "number (optional)",
  "currency": "string (optional)",
  "stock": "number (optional)",
```

```
"categoryId": "uuid (optional)",  
"images": ["array (optional)"],  
"attributes": "object (optional)"  
}
```

Response : 200 - Produit complet mis à jour

Erreurs : 400 (validation), 403 (not owner or admin), 404 (not found)

Autorisation : Token requis, vendeur propriétaire ou admin

DELETE /products/:productId

Description : Désactivation produit (soft delete)

Params : productId (uuid, required)

Body : none

Response : 204 No Content

Erreurs : 403 (not owner or admin), 404 (not found)

Autorisation : Token requis, vendeur propriétaire ou admin

POST /products/:productId/images

Description : Ajouter images produit

Params : productId (uuid, required)

Body :

```
{  
"images": ["array<string> (required, URLs)"]  
}
```

Response : 200 -

```
{  
"productId": "uuid",  
"images": ["array"]  
}
```

Erreurs : 400 (validation), 403, 404

Autorisation : Token requis, vendeur propriétaire ou admin

1.4 Catégories Produits [NoSQL MongoDB - catalog:3002]

GET /categories

Description : Liste hiérarchique catégories

Query : parentId=uuid, page=0, limit=50

Body : none

Response : 200 -

```
{
  "categories": [
    {
      "categoryId": "uuid",
      "name": "string",
      "parentId": "uuid|null",
      "childrenCount": number
    }
  ],
  "total": number
}
```

Erreurs : 400 (invalid filters)

Autorisation : Aucune (public)

GET /categories/:categoryId

Description : Détails catégorie + produits

Params : categoryId (uuid, required)

Query : page=0, limit=20

Body : none

Response : 200 -

```
{
  "categoryId": "uuid",
  "name": "string",
  "parentId": "uuid|null",
  "description": "string",
  "products": ["array"],
  "totalProducts": number
}
```

Erreurs : 404 (not found)

Autorisation : Aucune (public)

1.5 Panier [NoSQL MongoDB - cart:3004]

GET /cart

Description : Panier utilisateur courant

Body : none

Response : 200 -

```
{
  "items": [
    {
      "productId": "uuid",
      "name": "string",
      "price": number,
      "currency": "string",
      "quantity": number,
      "total": number
    }
  ],
  "totalItems": number,
  "totalAmount": number,
  "currency": "string"
}
```

Erreurs : 401 (no token)

Autorisation : Token requis

POST /cart/items

Description : Ajouter/modifier article panier

Body :

```
{
  "productId": "uuid (required)",
  "quantity": "number (required, 1-99)"
}
```

}

Response : 200 - Panier complet mis à jour

Erreurs : 400 (invalid quantity), 401, 404 (product not found/available)

Autorisation : Token requis

PUT /cart/items/:productId

Description : Modifier quantité article

Params : productId (uuid, required)

Body :

{

"quantity": "number (required, 0-99, 0=suppression)"

}

Response : 200 - Panier complet

Erreurs : 400, 401, 404

Autorisation : Token requis

DELETE /cart/items/:productId

Description : Supprimer article panier

Params : productId (uuid, required)

Body : none

Response : 204 No Content

Erreurs : 401, 404 (not in cart)

Autorisation : Token requis

DELETE /cart

Description : Vider panier complet

Body : none

Response : 204 No Content

Erreurs : 401

Autorisation : Token requis

1.6 Commandes [NoSQL MongoDB - orders:3003]

POST /orders

Description : Créer commande depuis panier

Body :

```
{  
  "shippingAddressId": "uuid (required)",  
  "billingAddressId": "uuid (required)",  
  "paymentMethod": "string (required, 'card'|'paypal'|'bank')",  
  "couponCode": "string (optional)"  
}
```

Response : 201 -

```
{  
  "orderId": "uuid",  
  "status": "pending",  
  "items": ["array"],  
  "shippingAddress": "object",  
  "billingAddress": "object",  
  "total": number,  
  "currency": "string",  
  "createdAt": "ISO date"  
}
```

Erreurs : 400 (cart empty/invalid addresses), 401, 402 (payment required)

Autorisation : Token requis

GET /orders

Description : Liste commandes utilisateur

Query : status=pending|confirmed|shipped|canceled, page=0, limit=20

Body : none

Response : 200 -

```
{  
  "orders": [  
    {  
      "orderId": "uuid",  
      "status": "string",  
      "total": number,
```

```
"currency": "string",  
"createdAt": "ISO date"  
}
```

```
],
```

```
"total": number  
}
```

Erreurs : 401

Autorisation : Token requis (client: ses commandes, admin: toutes)

GET /orders/:orderId

Description : Détail commande complète

Params : orderId (uuid, required)

Body : none

Response : 200 -

```
{  
  "orderId": "uuid",  
  "status": "string",  
  "userId": "uuid",  
  "items": [  
    {  
      "productId": "uuid",  
      "name": "string",  
      "quantity": number,  
      "price": number  
    }  
  ],  
  "shippingAddress": "object",  
  "billingAddress": "object",  
  "total": number,  
  "paymentStatus": "string",  
  "trackingNumber": "string|null"  
}
```

Erreurs : 403 (not owner/admin), 404

Autorisation : Token requis

PUT /orders/:orderId/cancel

Description : Annulation commande (avant paiement)

Params : orderId (uuid, required)

Body :

```
{  
  "reason": "string (optional)"  
}
```

Response : 200 -

```
{  
  "orderId": "uuid",  
  "status": "canceled"  
}
```

Erreurs : 400 (cannot cancel current status), 403 (not owner/admin), 404

Autorisation : Token requis (propriétaire ou admin)

PUT /orders/:orderId/status

Description : Changer statut (admin uniquement)

Params : orderId (uuid, required)

Body :

```
{  
  "status": "string (required, 'confirmed'|'shipped'|'delivered')"  
}
```

Response : 200 - Commande mise à jour

Erreurs : 400 (invalid transition), 403 (not admin), 404

Autorisation : Token requis, rôle admin

1.7 Paiements [NoSQL MongoDB - payments:3005]

POST /payments

Description : Initialiser paiement commande

Body :

```
{  
  "orderId": "uuid (required)",  
  "provider": "string (required, 'stripe'|'paypal')",  
  "paymentMethod": "string (optional)"  
}
```

Response : 201 -

```
{  
  "paymentId": "uuid",  
  "orderId": "uuid",  
  "status": "pending",  
  "provider": "string",  
  "amount": number,  
  "currency": "string",  
  "redirectUrl": "string"  
}
```

Erreurs : 400 (invalid order), 402 (payment required), 404

Autorisation : Token requis

GET /payments/:paymentId

Description : Statut paiement

Params : paymentId (uuid, required)

Body : none

Response : 200 -

```
{  
  "paymentId": "uuid",  
  "orderId": "uuid",  
  "status": "pending|success|failed",  
  "provider": "string",  
  "amount": number,  
  "currency": "string",  
  "transactionId": "string|null"  
}
```

Erreurs : 403 (not owner/admin), 404

Autorisation : Token requis

POST /payments/:paymentId/confirm

Description : Confirmer paiement externe (webhook/callback)

Params : paymentId (uuid, required)

Body :

```
{  
  "transactionId": "string (required)",  
  "status": "success|failed (required)"  
}
```

Response : 200 - Paiement mis à jour

Erreurs : 400 (invalid data), 403, 404

Autorisation : Token interne uniquement (webhook)

1.8 Adresses [NoSQL MongoDB - addresses:3006]

POST /addresses

Description : Ajouter adresse utilisateur

Body :

```
{  
  "type": "string (required, 'billing' | 'shipping')",  
  "line1": "string (required)",  
  "line2": "string (optional)",  
  "city": "string (required)",  
  "postalCode": "string (required)",  
  "country": "string (required, ISO)",  
  "isDefault": "boolean (optional)"  
}
```

Response : 201 -

```
{  
  "addressId": "uuid",  
  "type": "string",
```



```
"line1": "string",  
"city": "string",  
"postalCode": "string",  
"country": "string",  
"isDefault": "boolean"  
}
```

Erreurs : 400 (validation), 401

Autorisation : Token requis

GET /addresses

Description : Liste adresses utilisateur

Query : type=billing|shipping

Body : none

Response : 200 -

```
{  
  "addresses": ["array<complete address objects>"]  
}
```

Erreurs : 401

Autorisation : Token requis

2. API Interne (Gateway → Microservices)

Headers internes : X-User-Id, X-User-Role, X-Request-Id

2.1 Users Service (Port 3001) [SQL]

- POST /internal/users
- GET /internal/users/:userId
- PUT /internal/users/:userId
- POST /internal/users/authenticate

2.2 Catalog Service (Port 3002) [NoSQL]

- POST /internal/products
- GET /internal/products
- GET /internal/products/:productId
- PUT /internal/products/:productId

2.3 Cart Service (Port 3004) [NoSQL]

- GET /internal/cart
- POST /internal/cart/items
- PUT /internal/cart/items/:productId
- DELETE /internal/cart/items/:productId

2.4 Orders Service (Port 3003) [NoSQL]

- POST /internal/orders
- GET /internal/orders
- GET /internal/orders/:orderId
- PUT /internal/orders/:orderId/status

2.5 Payments Service (Port 3005) [NoSQL]

- POST /internal/payments
- GET /internal/payments/:paymentId
- POST /internal/payments/:paymentId/confirm

2.6 Addresses Service (Port 3006) [NoSQL]

- POST /internal/addresses
- GET /internal/addresses
- PUT /internal/addresses/:addressId
- DELETE /internal/addresses/:addressId

3. Architecture Base de Données

3.1 Mapping Services/Bases

Service	Port	Base	Justification
-----	-----	-----	-----
users	3001	PostgreSQL	ACID, rôles, transactions
catalog	3002	MongoDB	Recherche full-text, scalabilité
cart	3004	MongoDB	Éphémère TTL 30j, haute volumétrie
orders	3003	MongoDB	Transactions critiques ACID
payments	3005	MongoDB	Conformité PCI-DSS, audit
addresses	3006	MongoDB	Relations users/commandes

Répartition : 1 services SQL + 5 services NoSQL = CONFORME PROJET

3.2 Schémas SQL - PostgreSQL (Données critiques)

Utilisateurs & Rôles

sql

```
CREATE TABLE roles (  
  id SERIAL PRIMARY KEY,  
  code VARCHAR(20) UNIQUE NOT NULL  
);  
  
CREATE TABLE users (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  username VARCHAR(30) UNIQUE NOT NULL,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  telephone VARCHAR(20),  
  created_at TIMESTAMP DEFAULT NOW(),  
  updated_at TIMESTAMP DEFAULT NOW(),  
  statut_compte VARCHAR(20) DEFAULT 'actif' CHECK (statut_compte IN ('actif', 'suspendu', 'inactif'))  
);  
  
CREATE TABLE user_roles (  
  user_id UUID REFERENCES users(id),  
  role_id INT REFERENCES roles(id),  
  PRIMARY KEY (user_id, role_id)  
);
```

3.3 Schémas NoSQL - MongoDB (5 services)

Collections Catalog Service (Port 3002)

```
````javascript  
// Collection categories
{
 _id: ObjectId,
 categoryId: "uuid",
```

```
name: "Smartphones",
parentId: null,
path: ["uuid_root", "uuid_electronique", "uuid_smartphones"],
description: "Mobile phones and accessories",
level: 2,
productCount: 45,
createdAt: ISODate()
}
```

```
// Collection products
```

```
{
 _id: ObjectId,
 productId: "uuid",
 name: "iPhone 15 Pro",
 description: "Latest Apple smartphone",
 price: 1299.99,
 currency: "EUR",
 stock: 50,
 categoryId: "uuid",
 sellerId: "uuid", // ref SQL users
 images: ["url1", "url2"],
 attributes: {
 color: "Titanium",
 storage: "256GB",

 brand: "Apple"
 },
 status: "active",
 avgRating: 4.5,
 reviewsCount: 127,
```

```
 createdAt: ISODate(),
 updatedAt: ISODate()
}
```

```
// Collection reviews
```

```
{
 _id: ObjectId,
 reviewId: "uuid",
 productId: "uuid",
 userId: "uuid", // ref SQL users
 rating: 5,
 comment: "Excellent product!",
 status: "published",
 createdAt: ISODate()
}
```

```
// Collection shops (BOUTIQUE)
```

```
{
 _id: ObjectId,
 shopId: "uuid",
 name: "TechStore Pro",
 description: "Electronics and gadgets",
 ownerId: "uuid", // ref SQL users
 status: "active",
 rating: 4.3,
 productCount: 156,
 createdAt: ISODate(),
 updatedAt: ISODate()
}
```

```
// Collection stock (STOCK)
```

```
{
 _id: ObjectId,
 stockId: "uuid",
 productId: "uuid",
 quantity: 50,
 reserved: 5, // pour commandes en cours
 available: 45,
 location: "Warehouse A",
 lastRestock: ISODate(),
 lowStockThreshold: 10,
 updatedAt: ISODate()
}
'''
```

#### Collections Cart Service (Port 3004)

```
```javascript
// Collection carts (TTL 30j)
{
  _id: ObjectId,
  userId: "uuid", // ref SQL users
  items: [
    {
      productId: "uuid",
      quantity: 2,
      price: 1299.99,
      addedAt: ISODate(),
      product: {
        name: "iPhone 15 Pro",
        images: ["url1"]
      }
    }
  ]
}
```

```
],
totalAmount: 2599.98,
currency: "EUR",
totalItems: 2,
updatedAt: ISODate(),
expiresAt: ISODate() // TTL index
}
...
```

Collections Orders Service (Port 3003)

```
````javascript
// Collection orders
{
 _id: ObjectId,
 orderId: "uuid",
 userId: "uuid", // ref SQL users
 status: "pending",
 items: [
 {
 productId: "uuid",
 name: "iPhone 15 Pro",
 quantity: 2,
 price: 1299.99,
 total: 2599.98
 }
],
 shippingAddress: {
 addressId: "uuid", // ref MongoDB addresses
 line1: "123 Main St",
 city: "Paris",
 postalCode: "75001",
 }
}
```

```
 country: "FR"
 },
 billingAddress: {
 addressId: "uuid",
 line1: "123 Main St",
 city: "Paris",
 postalCode: "75001",
 country: "FR"
 },
 total: 2599.98,
 currency: "EUR",
 paymentStatus: "pending",
 trackingNumber: null,
 createdAt: ISODate(),
 updatedAt: ISODate()
}
```

// Collection orderStatus

```
{
 _id: ObjectId,
 orderId: "uuid",
 status: "confirmed",
 timestamp: ISODate(),
 comment: "Order confirmed by admin"
}
```

// Collection deliveries (LIVRAISON)

```
{
 _id: ObjectId,
 deliveryId: "uuid",
 orderId: "uuid",
```



```
addressId: "uuid",
status: "preparing", // preparing|shipped|in_transit|delivered
trackingNumber: "1Z999AA1234567890",
carrier: "chronopost",
estimatedDelivery: ISODate(),
actualDelivery: null,
createdAt: ISODate(),
updatedAt: ISODate()
}
```

```
// Collection returns (RETOUR)
```

```
{
 _id: ObjectId,
 returnId: "uuid",
 orderId: "uuid",
 productId: "uuid",
 reason: "defective",
 status: "requested", // requested|approved|rejected|completed
 refundAmount: 1299.99,
 refundStatus: "pending",
 createdAt: ISODate(),
 updatedAt: ISODate()
}
```

```
...
```

```
Collections Payments Service (Port 3005)
```

```
```javascript
```

```
// Collection payments
```

```
{
  _id: ObjectId,
  paymentId: "uuid",
```

```
orderId: "uuid", // ref MongoDB orders
userId: "uuid", // ref SQL users
status: "pending",
provider: "stripe",
amount: 2599.98,
currency: "EUR",
paymentMethod: "card",
transactionId: null,
redirectUrl: "https://checkout.stripe.com/pay/...",
metadata: {
  stripePaymentIntentId: "pi_1234567890"
},
createdAt: ISODate(),
updatedAt: ISODate()
}
```

```
// Collection paymentLogs
```

```
{
  _id: ObjectId,
  paymentId: "uuid",
  event: "payment_initiated",
  status: "pending",
  data: {
    gatewayResponse: "..."
  },
  timestamp: ISODate()
}
```

```
// Collection coupons (COUPON)
```

```
{
  _id: ObjectId,
```

```
couponId: "uuid",
code: "SUMMER2024",
discount: 15, // percentage
discountType: "percentage", // percentage|fixed
maxUses: 1000,
usedCount: 234,
minOrderAmount: 100,
validFrom: ISODate(),
validUntil: ISODate(),
isActive: true,
createdAt: ISODate(),
updatedAt: ISODate()
}
...
```

Collections Addresses Service (Port 3006)

```
``javascript
// Collection addresses
{
  _id: ObjectId,
  addressId: "uuid",
  userId: "uuid", // ref SQL users
  type: "shipping", // billing|shipping
  line1: "123 Main St",
  line2: "Apt 4B",
  city: "Paris",
  postalCode: "75001",
  country: "FR",
  isDefault: true,
  coordinates: {
    lat: 48.8566,
```

```

    lng: 2.3522
  },
  createdAt: ISODate(),
  updatedAt: ISODate()
}

// Collection addressValidation
{
  _id: ObjectId,
  addressId: "uuid",
  isValid: true,
  validationService: "google_maps",
  validatedAt: ISODate()
}
...

```

4. Flux Métier Exemple : Achat Complet

1. POST /auth/login [SQL] → JWT token
2. GET /products [NoSQL] → Recherche produits
3. POST /cart/items [NoSQL] → Ajout panier
4. POST /addresses [NoSQL] → Ajout adresse livraison
5. POST /orders [NoSQL] → Créer commande (réserve stock)
6. POST /payments [NoSQL] → Initie paiement Stripe
7. [Redirect] → Stripe checkout
8. POST /payments/:id/confirm [NoSQL] → Webhook finalise
9. PUT /orders/:id/status [NoSQL] → "confirmed"

5. Conformité Projet

Architecture microservices indépendants

- 1 service SQL pour données critiques (users/roles)

5 services NoSQL pour flexibilité et scalabilité

Communication réseau entre services

Persistance hybride SQL/NoSQL justifiée

Frontières claires entre services

API Gateway centralisée

JWT authentication standard

CONFORME AUX EXIGENCES PROJET